

Lab 4: Beat the Instructor

Multi-Robot Task Allocation for RoboEats Café

Saief Eddine BRAHMI

1 Introduction

This report presents my solution for the RoboEats Café coordination problem, where two service robots must efficiently deliver orders to multiple tables and return to the kitchen. The objective is to minimize the performance metric:

$$\text{Performance} = (\text{dist}_{R1} + \text{dist}_{R2}) \times \text{time} \quad (1)$$

where dist_{R1} and dist_{R2} are the distances traveled by each robot, and time is the total simulation time.

2 Solution Approach

My solution addresses two main challenges:

1. **Task Allocation:** Deciding which robot serves which tables
2. **Route Optimization:** Finding the optimal visiting order for each robot

2.1 Task Allocation Strategy

I use a **geographic partitioning** approach that splits tables based on their position in the café:

- **Robot 1** (starting at $[7, 1.75]$): Handles tables on the right side (Tables 5-10, with $x \geq 5$)
- **Robot 2** (starting at $[7, 1]$): Handles tables on the left side (Tables 1-4, with $x < 5$)

This strategy ensures both robots work in parallel, significantly reducing the total simulation time. While the total distance may increase compared to using a single robot, the time reduction results in a better overall performance metric.

2.2 Route Optimization (TSP)

For each robot's assigned tables, I solve a Traveling Salesman Problem (TSP) to find the optimal visiting order. Given the small number of tables (≤ 10), I use a brute-force approach that evaluates all permutations:

$$\text{Best Route} = \arg \min_{\pi} \sum_{i=0}^n d(\pi_i, \pi_{i+1}) \quad (2)$$

where π is a permutation of the assigned tables, and $d(a, b)$ is the Euclidean distance between points a and b .

2.3 Collision Avoidance

To prevent robots from colliding when returning to the kitchen, I added intermediate waypoints before the final destination. Each robot approaches from its designated side:

- Robot 1 passes through $[7, 2]$ before reaching its kitchen position
- Robot 2 passes through $[7, 1]$ before reaching its kitchen position

This simple strategy ensures smooth parallel return without path conflicts.

3 Implementation

I implemented the solution in `my_solver.py` with the following key functions:

- `distance(p1, p2)`: Computes Euclidean distance between two points
- `solve_tsp(start, tables, end)`: Finds optimal route using brute-force permutation search
- `find_balanced_allocation(tables)`: Splits tables geographically between robots
- `call(tables_to_serve)`: Main entry point that returns waypoints for both robots

4 Results

I tested the solution on the provided test cases:

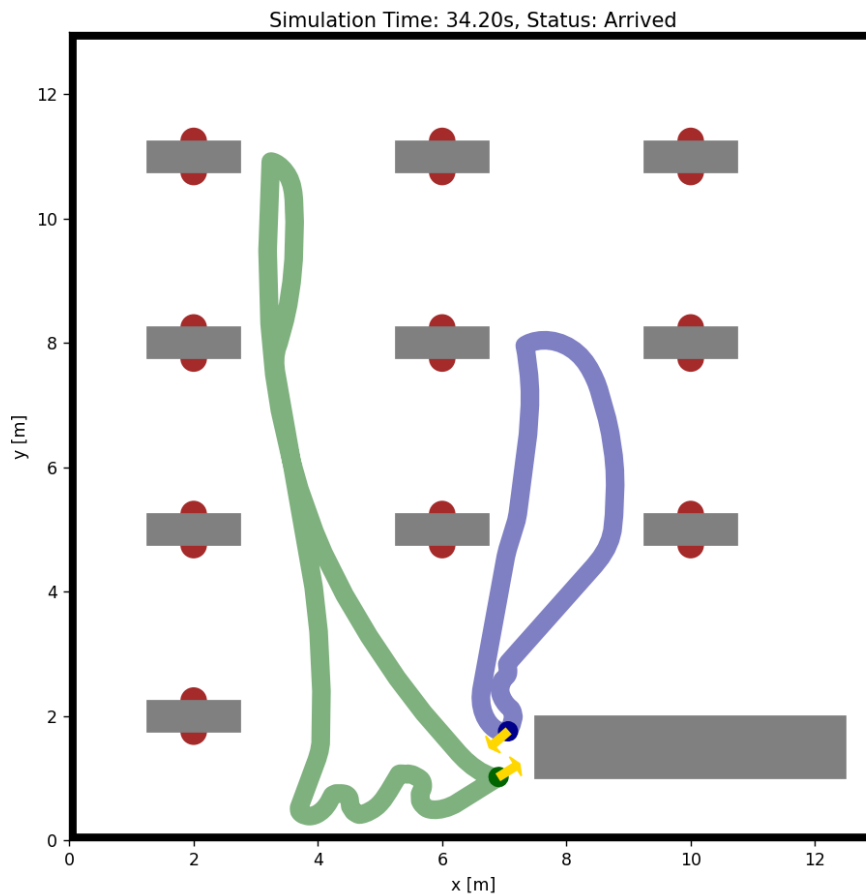


Figure 1: Simulation showing both robot trajectories in the RoboEats Café

```

PS C:\Users\brahm\Desktop\Lab4_cps> python lab4.py 4 6 8 3
2026-01-25 13:03 | INFO | Simulation environment 'RoboEatsCafe_config' started. Step time 0.100 s.
The list of tables to serve is: [4, 6, 8, 3]
R1: [6, 8] (13.30m) | R2: [4, 3] (21.62m)
2026-01-25 13:03 | INFO | pause the environment
2026-01-25 13:03 | INFO | Simulation Environment 'RoboEatsCafe_config' closing in 3.00 seconds.
2026-01-25 13:03 | INFO | Simulation Environment 'RoboEatsCafe_config' ended. Total time 34.20 seconds.
Trajectory length R1: 14.33 meters
Trajectory length R2: 26.34 meters
Time spent: 34.20 seconds
PS C:\Users\brahm\Desktop\Lab4_cps>

```

Figure 2: Terminal output showing performance metrics

Test Case	R1 Distance	R2 Distance	Time	Metric
4 6 8 3	14.33 m	26.34 m	34.20 s	1391.11
7 9 2	20.07 m	15.15 m	23.80 s	837.63
7 9	19.98 m	8.26 m	22.00 s	621.28
1 4 7 9 2	20.22 m	28.31 m	37.80 s	1834.43

Table 1: Performance results on test cases (all runs completed without collisions)

5 Conclusion

My solution demonstrates that parallel execution with two robots provides better performance than sequential delivery with a single robot. The geographic partitioning approach ensures minimal path overlap between robots while maximizing parallel work. The TSP optimization guarantees that each robot follows the shortest possible route for its assigned tables.

All test cases completed successfully without any robot collisions.

5.1 Key Insights

- Using both robots reduces time significantly, which dominates the performance metric
- Geographic partitioning prevents robots from crossing paths frequently
- Brute-force TSP is feasible given the small problem size (≤ 10 tables)
- Adding intermediate waypoints before returning to kitchen prevents collision conflicts